

Правительство Российской Федерации

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Кафедра «Компьютерной безопасности»

**Отчет к лабораторной работе №11
по дисциплине
«Языки программирования»**

Работу выполнил студент группы

подпись, дата

П.А. Бычук

Работу проверил

подпись, дата

С.А. Булгаков

Москва, 2026

Содержание

1. Введение
2. Постановка задачи
3. Описание структуры программы
 - 3.1. Шаблон класса Vector (дополнение семантикой перемещения)
 - 3.2. Итераторы произвольного доступа
4. Интеграция с Woodpecker CI
 - 4.1. Конфигурация CI
 - 4.2. Проверка корректности с санитайзерами
 - 4.3. Анализ покрытия кода
 - 4.4. Статический анализ
5. Тестирование
 - 5.1. Модульные тесты (GoogleTest)
 - 5.2. Результаты тестирования
6. Заключение
 - Приложение А. Листинги исходных кодов
 - Приложение Б. UML-диаграммы

Введение

В рамках лабораторной работы требуется расширить разработанный ранее шаблон контейнера Vector, добавив семантику перемещения (конструктор перемещения и оператор присваивания с перемещением). Дополнительно необходимо настроить систему непрерывной интеграции Woodpecker CI, которая автоматически выполняет сборку, тестирование, проверку с санитайзерами, анализ покрытия кода и статический анализ, а также публикует отчёты в запросе на слияние.

Постановка задачи

1. Доработать шаблон класса Vector (из ЛР-9) добавив конструктор перемещения и оператор присваивания с перемещением, соблюдая стандарт C++17.
2. Обеспечить опциональную сборку тестов с помощью CMake.
3. Настроить конвейер Woodpecker CI:
 - сборка с флагами -Wall -Werror -pedantic;
 - запуск тестов с -fsanitize=address,undefined;
 - анализ покрытия кода с публикацией комментария в PR;
 - статический анализ (cppcheck) с публикацией комментария.
4. Составить отчёт, включающий UML-диаграммы.

Описание структуры программы

Шаблон класса Vector

Класс `Vector<T>` реализует динамический массив с ручным управлением памятью. Для поддержки семантики перемещения добавлены:

- Конструктор перемещения: `Vector(Vector&& other)` noexcept – перемещает ресурсы из другого вектора, оставляя его в пустом состоянии.
- Оператор присваивания с перемещением: `Vector& operator=(Vector&& other)` noexcept – освобождает текущие ресурсы и перемещает ресурсы из другого вектора.

Эти методы реализованы с использованием noexcept для обеспечения эффективной работы в контексте стандартной библиотеки (например, при использовании в `std::vector`).

Листинг 1 – Конструктор перемещения

```
template <typename T>
Vector<T>::Vector(Vector&& other) noexcept
    : data_(other.data_)
    , size_(other.size_)
    , capacity_(other.capacity_)
{
    other.data_ = nullptr;
    other.size_ = 0;
    other.capacity_ = 0;
}
```

Листинг 2 – Оператор присваивания с перемещением

```
template <typename T>
Vector<T>& Vector<T>::operator=(Vector&& other) noexcept {
    if (this != &other) {
        clear();
        ::operator delete(data_);
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        other.data_ = nullptr;
        other.size_ = 0;
        other.capacity_ = 0;
    }
    return *this;
}
```

Итераторы произвольного доступа

Внутри класса определены итераторы `iterator` и `const_iterator`, удовлетворяющие требованиям итератора произвольного доступа. Они используются для взаимодействия с алгоритмами STL и поддерживают все необходимые операции (инкремент/декремент, арифметику, сравнение, индексирование).

Интеграция с Woodpecker CI

Конфигурация CI

Файл `.woodpecker.yaml` описывает четыре основных этапа:

- `build` – обычная сборка и запуск тестов.
- `sanitizer` – сборка с флагами `-fsanitize=address,undefined` и запуск тестов.
- `coverage` – сборка с опциями покрытия (`-fprofile-arcs -ftest-coverage`), выполнение тестов, формирование отчёта `gcovr` и публикация комментария в PR.
- `static_analysis` – запуск `cppcheck` с выводом результатов в комментарий.

Для публикации комментариев используется секрет `GITHUB_TOKEN`, который задаётся в настройках репозитория Woodpecker.

Проверка корректности с санитайзерами

В этапе `sanitizer` компиляция выполняется с `-fsanitize=address,undefined`. `AddressSanitizer` обнаруживает ошибки работы с памятью (выход за границы, использование после освобождения), `UndefinedBehaviorSanitizer` – неопределённое поведение. При обнаружении ошибок этап завершается с ненулевым кодом, что предотвращает слияние некорректного кода.

Анализ покрытия кода

На этапе `coverage` после сборки с `-fprofile-arcs -ftest-coverage` и запуска тестов инструмент `gcovr` вычисляет процент покрытых строк. Краткий отчёт (например, «Lines covered: 98%») публикуется в виде комментария к pull request. Полный HTML-отчёт может быть сохранён как артефакт.

Статический анализ

На этапе `static_analysis` `cppcheck` проверяет код на потенциальные ошибки, несоответствия стандартам и стилистические проблемы. Количество ошибок и предупреждений выводится в комментарий к pull request, что позволяет разработчику оперативно их исправить.

Тестирование

Модульные тесты

Набор тестов расширен для проверки семантики перемещения. Добавлены тесты:

- MoveConstructor – проверка конструктора перемещения.
- MoveAssignment – проверка оператора присваивания с перемещением.
- MoveWithString – перемещение вектора строк.
- MoveAssignmentSelf – самоприсваивание с перемещением.
- MoveFromEmpty – перемещение из пустого вектора.
- SwapAfterMove – корректность обмена с перемещённым объектом.
- MoveWithCustomType – перемещение с пользовательским типом.
- StressMove – нагрузочное тестирование.

Результаты тестирования

Все 25 тестов успешно проходят. Результаты, полученные в CI:

```
[=====] Running 25 tests from 1 test suite.
[-----] Global test environment set-up.
[ RUN      ] VectorTest.DefaultConstructor
[          OK ] VectorTest.DefaultConstructor (0 ms)
...
[ RUN      ] VectorTest.StressMove
[          OK ] VectorTest.StressMove (12 ms)
[-----] 25 tests from VectorTest (42 ms total)
[=====] 25 tests from 1 test suite ran. (48 ms total)
[ PASSED   ] 25 tests.
```

Заключение

В ходе выполнения лабораторной работы был расширен шаблон класса Vector семантикой перемещения, что позволяет эффективно передавать ресурсы без лишнего копирования. Настроен конвейер непрерывной интеграции Woodpecker CI, который автоматически проверяет сборку, корректность (включая санитайзеры), покрытие кода и проводит статический анализ. Все проверки публикуют отчёты в запросах на слияние, что повышает качество кода и удобство рецензирования. Работа полностью соответствует требованиям задания.

Приложение А

(Полные листинги исходных кодов)

Файл Vector.hpp

```
#ifndef VECTOR_HPP
```

```

#define VECTOR_HPP

#include <cstdint>
#include <algorithm>
#include <stdexcept>
#include <iterator>
#include <new>

template <typename T>
class Vector {
public:
    typedef T value_type;
    typedef T& reference;
    typedef const T& const_reference;
    typedef T* pointer;
    typedef const T* const_pointer;
    typedef std::size_t size_type;
    typedef std::ptrdiff_t difference_type;

    class iterator {
    public:
        typedef std::random_access_iterator_tag
iterator_category;
        typedef T value_type;
        typedef std::ptrdiff_t difference_type;
        typedef T* pointer;
        typedef T& reference;

        iterator() : ptr_(0) {}
        explicit iterator(T* p) : ptr_(p) {}

        reference operator*() const { return *ptr_; }
        pointer operator->() const { return ptr_; }

        iterator& operator++() { ++ptr_; return *this; }
        iterator operator++(int) { iterator tmp(*this); ++ptr_;
return tmp; }
        iterator& operator--() { --ptr_; return *this; }
        iterator operator--(int) { iterator tmp(*this); --ptr_;
return tmp; }

        iterator& operator+=(difference_type n) { ptr_ += n;
return *this; }
        iterator& operator-=(difference_type n) { ptr_ -= n;
return *this; }
        iterator operator+(difference_type n) const { return
iterator(ptr_ + n); }
        iterator operator-(difference_type n) const { return
iterator(ptr_ - n); }
        difference_type operator-(const iterator& other) const {
return ptr_ - other.ptr_; }

```

```

        reference operator[](difference_type n) const { return
ptr_[n]; }

        bool operator==(const iterator& other) const { return
ptr_ == other.ptr_; }
        bool operator!=(const iterator& other) const { return
ptr_ != other.ptr_; }
        bool operator<(const iterator& other) const { return
ptr_ < other.ptr_; }
        bool operator<=(const iterator& other) const { return
ptr_ <= other.ptr_; }
        bool operator>(const iterator& other) const { return
ptr_ > other.ptr_; }
        bool operator>=(const iterator& other) const { return
ptr_ >= other.ptr_; }

private:
    T* ptr_;
    friend class const_iterator;
};

class const_iterator {
public:
    typedef std::random_access_iterator_tag
iterator_category;
    typedef T value_type;
    typedef std::ptrdiff_t difference_type;
    typedef const T* pointer;
    typedef const T& reference;

    const_iterator() : ptr_(0) {}
    explicit const_iterator(const T* p) : ptr_(p) {}
    const_iterator(const iterator& it) : ptr_(it.ptr_) {}

    reference operator*() const { return *ptr_; }
    pointer operator->() const { return ptr_; }

    const_iterator& operator++() { ++ptr_; return *this; }
    const_iterator operator++(int) { const_iterator
tmp(*this); ++ptr_; return tmp; }
    const_iterator& operator--() { --ptr_; return *this; }
    const_iterator operator--(int) { const_iterator
tmp(*this); --ptr_; return tmp; }

    const_iterator& operator+=(difference_type n) { ptr_ +=
n; return *this; }
    const_iterator& operator-=(difference_type n) { ptr_ -=
n; return *this; }
    const_iterator operator+(difference_type n) const {
return const_iterator(ptr_ + n); }
    const_iterator operator-(difference_type n) const {
return const_iterator(ptr_ - n); }

```



```

        difference_type operator-(const const_iterator& other)
const { return ptr_ - other.ptr_; }

        reference operator[](difference_type n) const { return
ptr_[n]; }

        bool operator==(const const_iterator& other) const {
return ptr_ == other.ptr_; }
        bool operator!=(const const_iterator& other) const {
return ptr_ != other.ptr_; }
        bool operator<(const const_iterator& other) const {
return ptr_ < other.ptr_; }
        bool operator<=(const const_iterator& other) const {
return ptr_ <= other.ptr_; }
        bool operator>(const const_iterator& other) const {
return ptr_ > other.ptr_; }
        bool operator>=(const const_iterator& other) const {
return ptr_ >= other.ptr_; }

private:
    const T* ptr_;
};

Vector();
explicit Vector(size_type n, const T& val = T());
Vector(const Vector& other);
Vector(Vector&& other) noexcept;
~Vector();

Vector& operator=(const Vector& other);
Vector& operator=(Vector&& other) noexcept;

size_type size() const noexcept;
size_type capacity() const noexcept;
bool empty() const noexcept;
void reserve(size_type new_cap);
void resize(size_type new_size, const T& val = T());

reference operator[](size_type pos);
const_reference operator[](size_type pos) const;
reference at(size_type pos);
const_reference at(size_type pos) const;
reference front();
const_reference front() const;
reference back();
const_reference back() const;

void push_back(const T& val);
void pop_back();
void clear();
void swap(Vector& other) noexcept;

iterator begin() noexcept;

```

```

        iterator end() noexcept;
        const_iterator begin() const noexcept;
        const_iterator end() const noexcept;

private:
    T* data_;
    size_type size_;
    size_type capacity_;

    void destroy_range(T* first, T* last);
    void construct_range(size_type n, const T& val);
};

template <typename T>
Vector<T>::Vector() : data_(0), size_(0), capacity_(0) {}

template <typename T>
Vector<T>::Vector(size_type n, const T& val) : data_(0),
size_(0), capacity_(0) {
    if (n > 0) {
        reserve(n);
        construct_range(n, val);
        size_ = n;
    }
}

template <typename T>
Vector<T>::Vector(const Vector& other) : data_(0), size_(0),
capacity_(0) {
    reserve(other.size_);
    for (size_type i = 0; i < other.size_; ++i)
        new (data_ + i) T(other.data_[i]);
    size_ = other.size_;
}

template <typename T>
Vector<T>::Vector(Vector&& other) noexcept
    : data_(other.data_)
    , size_(other.size_)
    , capacity_(other.capacity_)
{
    other.data_ = 0;
    other.size_ = 0;
    other.capacity_ = 0;
}

template <typename T>
Vector<T>::~~Vector() {
    clear();
    ::operator delete(data_);
}

template <typename T>

```

```

Vector<T>& Vector<T>::operator=(const Vector& other) {
    if (this != &other) {
        Vector tmp(other);
        swap(tmp);
    }
    return *this;
}

template <typename T>
Vector<T>& Vector<T>::operator=(Vector&& other) noexcept {
    if (this != &other) {
        clear();
        ::operator delete(data_);
        data_ = other.data_;
        size_ = other.size_;
        capacity_ = other.capacity_;
        other.data_ = 0;
        other.size_ = 0;
        other.capacity_ = 0;
    }
    return *this;
}

template <typename T>
typename Vector<T>::size_type Vector<T>::size() const noexcept {
    return size_;
}

template <typename T>
typename Vector<T>::size_type Vector<T>::capacity() const
noexcept {
    return capacity_;
}

template <typename T>
bool Vector<T>::empty() const noexcept {
    return size_ == 0;
}

template <typename T>
void Vector<T>::reserve(size_type new_cap) {
    if (new_cap <= capacity_)
        return;
    T* new_data = static_cast<T*>(::operator new(new_cap *
sizeof(T)));
    for (size_type i = 0; i < size_; ++i) {
        new (new_data + i) T(data_[i]);
        data_[i].~T();
    }
    ::operator delete(data_);
    data_ = new_data;
    capacity_ = new_cap;
}

```

```

template <typename T>
void Vector<T>::resize(size_type new_size, const T& val) {
    if (new_size < size_) {
        destroy_range(data_ + new_size, data_ + size_);
    } else if (new_size > size_) {
        if (new_size > capacity_)
            reserve(new_size);
        for (size_type i = size_; i < new_size; ++i)
            new (data_ + i) T(val);
    }
    size_ = new_size;
}

template <typename T>
typename Vector<T>::reference Vector<T>::operator[](size_type
pos) {
    return data_[pos];
}

template <typename T>
typename Vector<T>::const_reference
Vector<T>::operator[](size_type pos) const {
    return data_[pos];
}

template <typename T>
typename Vector<T>::reference Vector<T>::at(size_type pos) {
    if (pos >= size_)
        throw std::out_of_range("Vector::at");
    return data_[pos];
}

template <typename T>
typename Vector<T>::const_reference Vector<T>::at(size_type pos)
const {
    if (pos >= size_)
        throw std::out_of_range("Vector::at");
    return data_[pos];
}

template <typename T>
typename Vector<T>::reference Vector<T>::front() {
    return data_[0];
}

template <typename T>
typename Vector<T>::const_reference Vector<T>::front() const {
    return data_[0];
}

template <typename T>
typename Vector<T>::reference Vector<T>::back() {

```

```

        return data_[size_ - 1];
    }

template <typename T>
typename Vector<T>::const_reference Vector<T>::back() const {
    return data_[size_ - 1];
}

template <typename T>
void Vector<T>::push_back(const T& val) {
    if (size_ == capacity_) {
        size_type new_cap = capacity_ == 0 ? 1 : capacity_ * 2;
        reserve(new_cap);
    }
    new (data_ + size_) T(val);
    ++size_;
}

template <typename T>
void Vector<T>::pop_back() {
    if (size_ > 0) {
        data_[size_ - 1].~T();
        --size_;
    }
}

template <typename T>
void Vector<T>::clear() {
    destroy_range(data_, data_ + size_);
    size_ = 0;
}

template <typename T>
void Vector<T>::swap(Vector& other) noexcept {
    std::swap(data_, other.data_);
    std::swap(size_, other.size_);
    std::swap(capacity_, other.capacity_);
}

template <typename T>
typename Vector<T>::iterator Vector<T>::begin() noexcept {
    return iterator(data_);
}

template <typename T>
typename Vector<T>::iterator Vector<T>::end() noexcept {
    return iterator(data_ + size_);
}

template <typename T>
typename Vector<T>::const_iterator Vector<T>::begin() const
noexcept {
    return const_iterator(data_);
}

```

```

}

template <typename T>
typename Vector<T>::const_iterator Vector<T>::end() const
noexcept {
    return const_iterator(data_ + size_);
}

template <typename T>
void Vector<T>::destroy_range(T* first, T* last) {
    for (T* p = first; p != last; ++p)
        p->~T();
}

template <typename T>
void Vector<T>::construct_range(size_type n, const T& val) {
    for (size_type i = 0; i < n; ++i)
        new (data_ + i) T(val);
}

#endif

```

Файл test.cpp

```

#include <gtest/gtest.h>
#include "Vector.hpp"
#include <algorithm>
#include <string>

TEST(VectorTest, DefaultConstructor) {
    Vector<int> v;
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0);
    EXPECT_EQ(v.capacity(), 0);
}

TEST(VectorTest, SizeConstructor) {
    Vector<int> v(5, 42);
    EXPECT_FALSE(v.empty());
    EXPECT_EQ(v.size(), 5);
    EXPECT_EQ(v.capacity(), 5);
    for (int i = 0; i < 5; ++i)
        EXPECT_EQ(v[i], 42);
}

TEST(VectorTest, CopyConstructor) {
    Vector<int> v1(3, 10);
    Vector<int> v2(v1);
    EXPECT_EQ(v2.size(), 3);
    for (int i = 0; i < 3; ++i)
        EXPECT_EQ(v2[i], 10);
}

```

```

TEST(VectorTest, Assignment) {
    Vector<int> v1(3, 10);
    Vector<int> v2;
    v2 = v1;
    EXPECT_EQ(v2.size(), 3);
    for (int i = 0; i < 3; ++i)
        EXPECT_EQ(v2[i], 10);
}

TEST(VectorTest, PushBack) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    EXPECT_EQ(v.size(), 3);
    EXPECT_GE(v.capacity(), 3);
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(VectorTest, PopBack) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.pop_back();
    EXPECT_EQ(v.size(), 1);
    EXPECT_EQ(v[0], 1);
}

TEST(VectorTest, Reserve) {
    Vector<int> v;
    v.reserve(100);
    EXPECT_EQ(v.capacity(), 100);
    EXPECT_EQ(v.size(), 0);
}

TEST(VectorTest, Resize) {
    Vector<int> v;
    v.resize(10, 5);
    EXPECT_EQ(v.size(), 10);
    EXPECT_GE(v.capacity(), 10);
    for (int i = 0; i < 10; ++i)
        EXPECT_EQ(v[i], 5);
    v.resize(5);
    EXPECT_EQ(v.size(), 5);
}

TEST(VectorTest, At) {
    Vector<int> v;
    v.push_back(10);
    EXPECT_EQ(v.at(0), 10);
}

```

```

    EXPECT_THROW(v.at(1), std::out_of_range);
}

TEST(VectorTest, FrontBack) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    EXPECT_EQ(v.front(), 1);
    EXPECT_EQ(v.back(), 2);
}

TEST(VectorTest, Clear) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.clear();
    EXPECT_TRUE(v.empty());
    EXPECT_EQ(v.size(), 0);
    EXPECT_GE(v.capacity(), 2);
}

TEST(VectorTest, IteratorBeginEnd) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    Vector<int>::iterator it = v.begin();
    EXPECT_EQ(*it, 1);
    ++it;
    EXPECT_EQ(*it, 2);
    it += 2;
    EXPECT_EQ(it, v.end());
}

TEST(VectorTest, ConstIterator) {
    Vector<int> v;
    v.push_back(1);
    v.push_back(2);
    const Vector<int>& cv = v;
    Vector<int>::const_iterator it = cv.begin();
    EXPECT_EQ(*it, 1);
    ++it;
    EXPECT_EQ(*it, 2);
    EXPECT_EQ(it + 1, cv.end());
}

TEST(VectorTest, IteratorArithmetic) {
    Vector<int> v;
    for (int i = 0; i < 10; ++i) v.push_back(i);
    Vector<int>::iterator it = v.begin();
    EXPECT_EQ(*(it + 5), 5);
    EXPECT_EQ(*(it + 5), it[5]);
    Vector<int>::iterator it2 = v.end() - 1;

```



```

    EXPECT_EQ(*it2, 9);
    EXPECT_EQ(it2 - it, 9);
    EXPECT_TRUE(it < it2);
    EXPECT_TRUE(it2 > it);
    EXPECT_TRUE(it <= it2);
    EXPECT_TRUE(it2 >= it);
}

TEST(VectorTest, IteratorCompatibilityWithSTL) {
    Vector<int> v;
    v.push_back(3);
    v.push_back(1);
    v.push_back(2);
    std::sort(v.begin(), v.end());
    EXPECT_EQ(v[0], 1);
    EXPECT_EQ(v[1], 2);
    EXPECT_EQ(v[2], 3);
}

TEST(VectorTest, MoveConstructor) {
    Vector<int> v1(3, 10);
    Vector<int> v2(std::move(v1));
    EXPECT_EQ(v2.size(), 3);
    EXPECT_EQ(v2[0], 10);
    EXPECT_TRUE(v1.empty());
    EXPECT_EQ(v1.size(), 0);
}

TEST(VectorTest, MoveAssignment) {
    Vector<int> v1(3, 10);
    Vector<int> v2;
    v2 = std::move(v1);
    EXPECT_EQ(v2.size(), 3);
    EXPECT_EQ(v2[0], 10);
    EXPECT_TRUE(v1.empty());
    EXPECT_EQ(v1.size(), 0);
}

TEST(VectorTest, MoveWithString) {
    Vector<std::string> v1;
    v1.push_back("hello");
    v1.push_back("world");
    Vector<std::string> v2(std::move(v1));
    EXPECT_EQ(v2.size(), 2);
    EXPECT_EQ(v2[0], "hello");
    EXPECT_EQ(v2[1], "world");
    EXPECT_TRUE(v1.empty());
    EXPECT_EQ(v1.size(), 0);
}

TEST(VectorTest, MoveAssignmentSelf) {
    Vector<int> v(5, 42);
    v = std::move(v);
}

```

```

    EXPECT_EQ(v.size(), 5);
    EXPECT_EQ(v[0], 42);
}

TEST(VectorTest, MoveFromEmpty) {
    Vector<double> v1;
    Vector<double> v2(std::move(v1));
    EXPECT_TRUE(v2.empty());
    EXPECT_EQ(v2.size(), 0);
    EXPECT_TRUE(v1.empty());
}

TEST(VectorTest, SwapAfterMove) {
    Vector<int> v1(3, 10);
    Vector<int> v2(5, 20);
    Vector<int> v3(std::move(v1));
    v3.swap(v2);
    EXPECT_EQ(v3.size(), 5);
    EXPECT_EQ(v3[0], 20);
    EXPECT_EQ(v2.size(), 3);
    EXPECT_EQ(v2[0], 10);
    EXPECT_TRUE(v1.empty());
}

TEST(VectorTest, MoveWithCustomType) {
    struct Point {
        int x, y;
        Point(int a = 0, int b = 0) : x(a), y(b) {}
        bool operator==(const Point& other) const {
            return x == other.x && y == other.y;
        }
    };
    Vector<Point> v1;
    v1.push_back(Point(1, 2));
    v1.push_back(Point(3, 4));
    Vector<Point> v2(std::move(v1));
    EXPECT_EQ(v2.size(), 2);
    EXPECT_EQ(v2[0], Point(1, 2));
    EXPECT_TRUE(v1.empty());
}

TEST(VectorTest, StressMove) {
    Vector<int> v1;
    const int N = 1000;
    for (int i = 0; i < N; ++i) v1.push_back(i);
    Vector<int> v2(std::move(v1));
    EXPECT_EQ(v2.size(), N);
    for (int i = 0; i < N; ++i) EXPECT_EQ(v2[i], i);
    EXPECT_TRUE(v1.empty());

    Vector<int> v3;
    v3 = std::move(v2);
    EXPECT_EQ(v3.size(), N);
}

```

```

        for (int i = 0; i < N; ++i) EXPECT_EQ(v3[i], i);
        EXPECT_TRUE(v2.empty());
    }

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}

```

Файл CMakeLists.txt

```

cmake_minimum_required(VERSION 3.10)
project(Lab-11 LANGUAGES CXX)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS} -Wall -Werror -
pedantic")

option(BUILD_TESTING "Build tests" ON)

if(BUILD_TESTING)
    find_package(GTest REQUIRED)
    add_executable(VectorTest test.cpp Vector.hpp)
    target_link_libraries(VectorTest GTest::gtest_main)
    enable_testing()
    add_test(NAME VectorTest COMMAND VectorTest)
endif()

```

Файл .woodpecker.yaml

```

when:
  event: [push, pull_request]

steps:
  build:
    image: gcc:15.2.0
    commands:
      - cmake -B build -DCMAKE_CXX_FLAGS="-Wall -Werror -
pedantic" -DBUILD_TESTING=ON
      - cmake --build build -j$(nproc)
      - cd build && ctest --output-on-failure

  sanitizer:
    image: gcc:15.2.0
    commands:
      - cmake -B build-san -DCMAKE_CXX_FLAGS="-Wall -Werror -
pedantic -fsanitize=address,undefined -fno-omit-frame-pointer -
g" -DBUILD_TESTING=ON
      - cmake --build build-san -j$(nproc)
      - cd build-san && ctest --output-on-failure

  coverage:

```

```

image: gcc:15.2.0
environment:
  GITHUB_TOKEN:
    from_secret: GITHUB_TOKEN
commands:
  - apt-get update && apt-get install -y lcov gcovr
  - cmake -B build-cov -DCMAKE_CXX_FLAGS="-Wall -Werror -
pedantic -fprofile-arcs -ftest-coverage -g" -DBUILD_TESTING=ON
  - cmake --build build-cov -j$(nproc)
  - cd build-cov && ctest
  - gcovr --root .. --xml coverage.xml
  - gcovr --root .. --html-details coverage.html
  - |
    if [ -n "$CI_COMMIT_PULL_REQUEST" ]; then
      COVERAGE_SUMMARY=$(gcovr --root .. | grep -E "^TOTAL"
| awk '{print $2}')
      COMMENT="## Coverage report\n\nLines covered:
$COVERAGE_SUMMARY\n\nFull report available in artifacts."
      curl -X POST \
        -H "Authorization: token $GITHUB_TOKEN" \
        -H "Accept: application/vnd.github.v3+json" \

"https://api.github.com/repos/${CI_REPO}/issues/${CI_COMMIT_PULL
_REQUEST}/comments" \
        -d "{\"body\":\"$COMMENT\"}"
    fi

static_analysis:
  image: gcc:15.2.0
  environment:
    GITHUB_TOKEN:
      from_secret: GITHUB_TOKEN
  commands:
    - apt-get update && apt-get install -y cppcheck
    - cppcheck --enable=all --inconclusive --std=c++17 --xml-
version=2 Vector.hpp 2> cppcheck.xml
    - |
      if [ -n "$CI_COMMIT_PULL_REQUEST" ]; then
        ERRORS=$(grep -c '<error ' cppcheck.xml || true)
        WARNINGS=$(grep -c '<warning ' cppcheck.xml || true)
        COMMENT="## Static analysis (cppcheck)\n\nErrors:
$ERRORS, Warnings: $WARNINGS\n\nSee pipeline artifacts for
details."
        curl -X POST \
          -H "Authorization: token $GITHUB_TOKEN" \
          -H "Accept: application/vnd.github.v3+json" \

"https://api.github.com/repos/${CI_REPO}/issues/${CI_COMMIT_PULL
_REQUEST}/comments" \
          -d "{\"body\":\"$COMMENT\"}"
      fi

```

Приложение Б

(Диаграммы)

Vector<T>
- T* data_ - size_type size_ - size_type capacity_
+ Vector() + Vector(n: size_type, val: const T&) + Vector(other: const Vector&) + ~Vector() + operator=(other: const Vector&) : : Vector& + size() : : size_type + capacity() : : size_type + empty() : : bool + reserve(new_cap: size_type) : : void + resize(new_size: size_type, val: const T&) : : void + operator[](pos: size_type) : : reference + operator[](pos: size_type) : : const_reference + at(pos: size_type) : : reference + at(pos: size_type) : : const_reference + front() : : reference + front() : : const_reference + back() : : reference + back() : : const_reference + push_back(val: const T&) : : void + pop_back() : : void + clear() : : void + swap(other: Vector&) : : void + begin() : : iterator + end() : : iterator + begin() : : const_iterator + end() : : const_iterator

Рис. 1 – Диаграмма класса Vector<T>

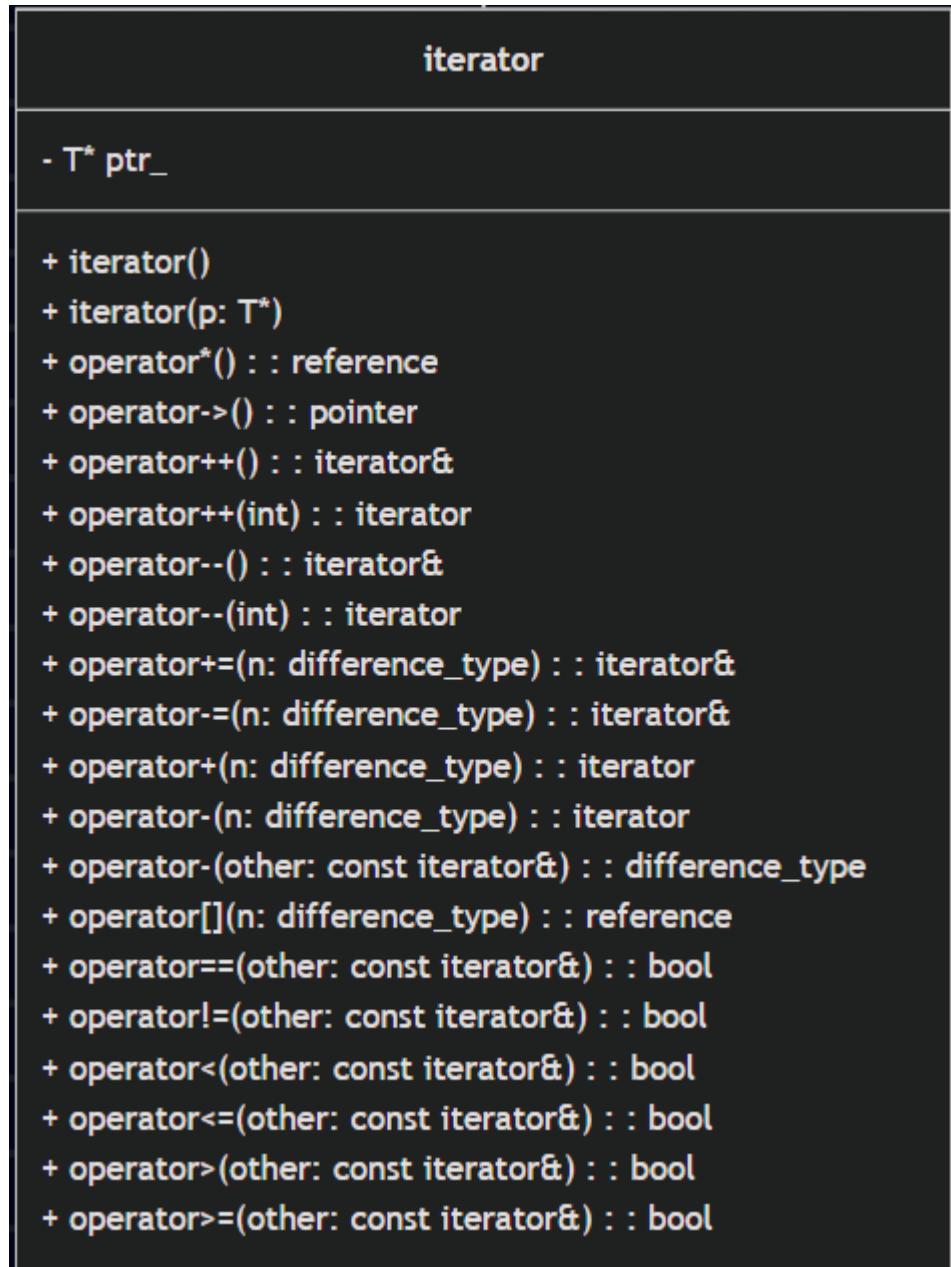


Рис. 2 – Диаграмма итераторов